Quality and Local Support of Trajectory Windows

Definitions, intuition and validation of the metrics used to test hypothesis H2

[Author names]

The hypothesis

GTA edits short windows of offline-RL data with a diffusion model, using **one** editing strength μ for every window.

Hypothesis H2: the best μ for a window depends on two of its properties

- **H2a, local support:** the better the region around the window is covered by the data, the **higher** μ can be (the model knows the region, so edits stay realistic).
- **H2b, quality:** the higher the window's own reward, the **lower** μ should be (a good window has less to gain).
- **H2c, downstream:** choosing μ per window improves TD3+BC trained on the edited data.

To test H2a and H2b we need two numbers per window: a **quality** Q and a **local support** S. Support is measured three ways: S_1, S_2, S_3.

What GTA edits: a window

Data: `halfcheetah-medium-v2`, 1,000 episodes $\times$ 1,000 steps $= 10^6$ transitions; state $s_t \in \mathbb{R}^{17}$, action $a_t \in \mathbb{R}^6$, reward r_t.

The window of episode e starting at step b:

$$W = \begin{bmatrix} s_b & a_b & r_b \\ \vdots & \vdots & \vdots \\ s_{b+30} & a_{b+30} & r_{b+30} \\ s_{b+31} & \mathbf{0} & 0 \end{bmatrix} \in \mathbb{R}^{32\times 24}$$

- 31 transitions + the final next-state; $24 = 17 + 6 + 1$ columns.
- Stride 1: 970 windows per episode, 970,000 in total.
- An edit adds noise σ to the whole window once, then the model removes it while asking for $1.3\times$ the window's return.

μ	0.1	0.2	0.25	0.3	0.35	0.4	0.5	0.75
noise σ added (data std ≈ 1)	0.015	0.081	0.16	0.32	0.54	0.98	2.43	17.3

Conventions shared by every metric

- **Unit:** one number per window i, identified by (episode e_i, start b_i).
- **Windows used:** starts 0–800 (801,000 windows); later starts are cut short by the episode end.
- **Pool:** $n = 20{,}000$ windows drawn uniformly at random (seed 0). S_3 (expensive) on a random 100 of them.
- **Percentile:** every raw metric is converted to its percentile in the pool,

$$\mathrm{pct}(u_i) = \frac{\mathrm{rank}(u_i)}{n} \in (0, 1].$$

- **Direction:** after conversion, **higher = better quality or more support**. Distances and errors (smaller is better) are negated first.

In plain words

Only the *order* of the windows is used. Percentiles make the metrics comparable and independent of their units.

Source quality Q

$$q_i = \sum_{t=0}^{30} \gamma^t \, r_{b_i+t}, \qquad \gamma = 0.99, \qquad Q_i = \mathrm{pct}(q_i).$$

Pool: 10th / 50th / 90th percentile of $q = 101.7$ / 133.9 / 151.6.

Why only the reward inside the window? The model is conditioned on the return-to-go to the episode end, $\mathrm{RTG}_i = \sum_{k \geq b_i} \gamma^{k-b_i} r_k$. RTG shrinks for late windows even when the behaviour is equally good, so it describes position in the episode, not behaviour.

+ exact, no estimation
+ directly interpretable: "reward this window earns"

– ignores what happens after the window
– correlated with support ($\rho \approx 0.42$–0.49): unusual windows earn less, so the study separates the two statistically

What “local support” means

In plain words

A window has **high support** when the dataset contains many similar situations (postures, velocities, actions); **low support** when it is unusual, e.g. the first steps of an episode, when the cheetah accelerates from rest (once per episode).

Why it should matter: the model learned from the data. Where data are dense it edits realistically; where sparse it guesses, and strong edits may break physics.

Support is a property of a distribution we never observe directly, so we estimate it three independent ways:

metric	what it looks at	cost
S_1	the **data**: how far away are similar transitions?	cheap
S_2	the **model**: how accurately can it remove noise from the window?	cheap
S_3	the **model**: exact probability density of the window	expensive

S_1: nearest-neighbour distance in the data

1. **Common scale.** Standardise every state–action pair $y_j = (s_j, a_j) \in \mathbb{R}^{23}$ over all $N = 10^6$ transitions:

$$z_{j,d} = \frac{y_{j,d} - \bar{y}_d}{\mathrm{sd}(y_{\cdot,d})}, \qquad d = 1, \ldots, 23.$$

2. **20th nearest neighbour from another episode**, for each step t of the window (transition j_t):

$$d_{20}(j_t) = \text{20th smallest of } \big\{ \|z_{j_t} - z_{j'}\|_2 : \ \text{episode}(j') \neq \text{episode}(j_t) \big\}.$$

3. **Summarise the window:**

$$\mathrm{s1_dist}_i = \text{median}_{t=0,\ldots,30}\ d_{20}(j_t), \qquad S_{1,i} = \mathrm{pct}\big(-\mathrm{s1_dist}_i\big).$$

Pool: s1_dist 10th / 50th / 90th percentile = 0.92 / 1.10 / 1.49 (range 0.69–10.5).

S_1: why a distance measures density

A ball of radius r around x in $\mathbb{R}^{23}$ contains about "density $\times$ volume" of the N points:

$$\frac{\#\{\text{points in the ball}\}}{N} \approx p(x)\, V_{23}\, r^{23}.$$

Grow the ball until it holds $k = 20$ points; its radius is then $d_{20}(x)$. Solving for $p(x)$:

$$\boxed{\hat{p}(x) = \frac{k}{N\, V_{23}\, d_k(x)^{23}}} \qquad \text{(the } k\text{-nearest-neighbour density estimate)}$$

For fixed k and N, $\hat{p}$ **falls** as d_k **rises**. Ranking by $-d_k$ is therefore identical to ranking by estimated density; since only ranks are used, V_{23} and the exponent never matter.

In plain words

"How far must I look to find 20 similar moments in other runs?" Short = crowded region, long = unusual window.

S_1: design choices, strengths, weaknesses

Design choices

- **Other episodes only:** at 20 steps/s consecutive steps are nearly identical; otherwise the 20 nearest would be the step's own neighbours in time (± 10 steps) for every window.
- $k = 20$**, not 1:** the single nearest neighbour is too sensitive to chance. 20 is a judgement call.
- **Median over steps:** one odd step should not decide the window.
- **Exact search** over all 10^6 transitions (FAISS on CPU or GPU matrix product; agree to 10^{-5}).

+ measures the data directly; no model
+ clear statistical meaning (density estimate)
+ inverse of the GTA paper's own "novelty" metric
+ simple to explain and reproduce

– scores steps separately: ignores their order
– all 23 dimensions weigh equally
– in 23-D distances bunch together (spread here still usable: $(p_{90} - p_{10})/\text{median} = 0.52$)
– k and the median are choices

Primer: what the diffusion model computes

- **Noisy version** of a normalised window $\boldsymbol{x} \in \mathbb{R}^{32\times 24}$: $\boldsymbol{x}_\sigma = \boldsymbol{x} + \sigma\varepsilon, \quad \varepsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Write p_σ for the distribution of $\boldsymbol{x}_\sigma$; p_0 is the data.
- **Denoiser** $D_\theta(\boldsymbol{x}_\sigma; \sigma, c)$, trained to recover $\boldsymbol{x}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{\boldsymbol{x},\sigma,\boldsymbol{\varepsilon}}\Big[\lambda(\sigma)\,\big\|D_\theta(\boldsymbol{x} + \sigma\boldsymbol{\varepsilon}; \sigma, c) - \boldsymbol{x}\big\|^2\Big].$$

  The condition c (the return) is dropped one time in four, so the network can also denoise **unconditionally** ($c = \varnothing$).
- **EDM wrapper** (Karras et al. 2022), with network F_θ:

$$D_\theta(\boldsymbol{x}; \sigma) = c_{\text{skip}}\,\boldsymbol{x} + c_{\text{out}}\,F_\theta\big(c_{\text{in}}\,\boldsymbol{x};\; c_{\text{noise}}\big),$$

$$c_{\text{skip}} = \frac{1}{\sigma^2+1}, \qquad c_{\text{out}} = \frac{\sigma}{\sqrt{\sigma^2+1}}, \qquad c_{\text{in}} = \frac{1}{\sqrt{\sigma^2+1}}, \qquad c_{\text{noise}} = \tfrac{1}{4}\ln\sigma.$$

Primer: the key fact linking denoising to density

Tweedie's formula

The best denoiser outputs the average clean window, $D^\star(\boldsymbol{x};\sigma) = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_\sigma = \boldsymbol{x}]$, and

$$\nabla_{\boldsymbol{x}} \log p_\sigma(\boldsymbol{x}) = \frac{D^\star(\boldsymbol{x};\sigma) - \boldsymbol{x}}{\sigma^2}.$$

It points towards where the data are dense: a trained denoiser is a **map of the data density** at every noise level.

S_2: how well the model reconstructs the window

1. Normalise window i: $\tilde{\mathbf{x}}_i$.
2. **Corrupt and repair**, unconditionally, for $\sigma \in \{0.16, 0.5, 2.4\}$ and $R = 4$ draws ε_n (one network call each):
$$\hat{\mathbf{x}}_{i,n}(\sigma) = D_\theta\big(\tilde{\mathbf{x}}_i + \sigma\varepsilon_n;\ \sigma,\ \varnothing\big).$$
3. **Error**, with mask M dropping the last row's always-zero action and reward (761 of 768 entries count, as in training):
$$e_i(\sigma) = \frac{1}{R}\sum_{n=1}^{R} \frac{\sum_{t,d} M_{td}\big(\hat{\mathbf{x}}_{i,n}(\sigma) - \tilde{\mathbf{x}}_i\big)^2_{td}}{\sum_{t,d} M_{td}}.$$
4. **Common scale** (pool medians of e: 0.0045, 0.022, 0.153, a 30× range):
$$\ell_i(\sigma) = \ln e_i(\sigma), \quad z_i(\sigma) = \frac{\ell_i(\sigma) - \mathrm{mean}_j\, \ell_j(\sigma)}{\mathrm{sd}_j\, \ell_j(\sigma)}, \quad \mathrm{s2_score}_i = -\frac{1}{3}\sum_{\sigma} z_i(\sigma).$$
5. $S_{2,i} = \mathrm{pct}(\mathrm{s2_score}_i)$. Small error ⇒ high support.

S_2: why reconstruction error measures support

- **Intuition:** where the model has seen many similar windows it undoes a corruption precisely; where it has seen few, its repair drifts towards what it knows. (Tweedie: the denoiser pulls points towards dense regions.)
- **Theory:** the model's negative log-likelihood is bounded by a noise-weighted integral of this error (Kingma et al. 2021; Song et al. 2021):

$$-\log p_\theta(\boldsymbol{x}) \leq \int_{\sigma_{\min}}^{\sigma_{\max}} w(\sigma)\, \mathbb{E}_{\boldsymbol{\varepsilon}}\left[\left\| D_\theta(\boldsymbol{x}+\sigma\boldsymbol{\varepsilon};\sigma)-\boldsymbol{x}\right\|^2\right] d\sigma + C.$$

  S_2 evaluates the integrand at 3 points: a cheap approximation of the log-likelihood (S_3).
- **Relevance:** the three σ are the noise an edit adds at $\mu \approx 0.25,\ 0.35,\ 0.5$.
- **No condition:** support should describe the window, not the return we will later ask for.
- **Logs and z-scores:** errors are skewed and differ multiplicatively; each σ gets an equal say.

S_2: strengths and weaknesses

Strengths

+ looks at the whole 32×24 window: step order, actions, rewards

+ measures the very model that edits, at the noise levels editing uses

+ linked in theory to the model's likelihood

+ cheap: 12 network calls per window, $\approx$ 3 s for 20,000 windows on a GPU

Weaknesses

– describes the **model**, not the data; the model was trained on these windows, so it partly reflects how well each region was learned

– equal weights for the 3 noise levels are a practical choice, not the bound's $w(\sigma)$

– only 4 draws per level: some randomness per window

– less transparent than S_1

S_3: exact log-likelihood, explained from scratch

S_2 *approximates* how likely the model finds a window; S_3 computes it *exactly*: $\log p_\theta(\boldsymbol{x})$, the density the trained model defines. Four ideas are needed.

Idea 1: an ODE is a rule for how a point moves

$$\frac{d\boldsymbol{x}}{d\sigma} = f(\boldsymbol{x}, \sigma)$$

means: when the noise level rises by a tiny $\Delta\sigma$, move $\boldsymbol{x}$ by $f(\boldsymbol{x}, \sigma)\,\Delta\sigma$. It is a velocity field.

- **Solving** it = repeating small moves: $\boldsymbol{x} \leftarrow \boldsymbol{x} + f(\boldsymbol{x}, \sigma)\Delta\sigma$ (Euler's method).
- We use **dopri5** (Dormand–Prince): it checks the velocity at several points per step and picks the step size so the error stays below 10^{-4}.

S_3, idea 2: a smooth path from each window to noise

For EDM diffusion models there is a special velocity field, the **probability-flow ODE** (Song et al. 2021):

$$\frac{d\boldsymbol{x}}{d\sigma} = f(\boldsymbol{x}, \sigma) = \frac{\boldsymbol{x} - D_\theta(\boldsymbol{x}; \sigma)}{\sigma}.$$

1. **Deterministic and reversible:** each window follows one fixed path; no randomness.
2. **Transports the whole distribution:** start with points from the data, let each follow the ODE to level σ; the cloud is then distributed exactly as p_σ.

At $\sigma_{\max} = 80$ the data (scale ≈ 1) are negligible, so $p_{80} = \mathcal{N}(\mathbf{0}, 80^2 \boldsymbol{I})$: a density we can write down exactly.

In plain words

Every window can be slid, smoothly and without randomness, into a big Gaussian cloud whose density we know. If we also know how much the sliding stretches space, we can work back to the window's density.

S_3, idea 3: densities change when space is stretched

One dimension. A map $z = 2x$ stretches every distance by 2. Probability is conserved, so the density halves:

$$p_X(x) = p_Z(z)\left|\frac{dz}{dx}\right| \quad \Longleftrightarrow \quad \log p_X(x) = \log p_Z(z) + \log\left|\frac{dz}{dx}\right|.$$

log-density at the start = log-density at the end + log of the stretch along the way.

Many dimensions: "stretch" = change of *volume* of a tiny box around the point.

Along an ODE path the volume changes continuously; its rate of change is the **divergence**

$$\nabla\cdot f(\boldsymbol{x},\sigma) = \sum_{m=1}^{768}\frac{\partial f_m}{\partial x_m}(\boldsymbol{x},\sigma) = \operatorname{tr}\left(\frac{\partial f}{\partial \boldsymbol{x}}(\boldsymbol{x},\sigma)\right),$$

the sum of the diagonal of the 768×768 Jacobian. Positive: neighbours move apart (space expands). Negative: they converge.

S_3: the log-likelihood formula

Ideas 2 and 3 together:

$$\log p_\theta(\boldsymbol{x}) = \underbrace{\log \mathcal{N}\big(\boldsymbol{x}(\sigma_{\max});\, \mathbf{0},\, \sigma_{\max}^2 \boldsymbol{I}\big)}_{\text{where the window lands in the Gaussian cloud}} + \underbrace{\int_{\sigma_{\min}}^{\sigma_{\max}} \nabla \cdot f\big(\boldsymbol{x}(\sigma), \sigma\big)\, d\sigma}_{\text{how much space expanded along its path}}$$

with $\boldsymbol{x}(\sigma_{\min}) = \tilde{\boldsymbol{x}}_i$, $\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$.

- The path starts at 0.002, not 0, because f divides by σ; the difference is a negligible blur.

In plain words

A window in a dense region sits in a tight crowd. Sliding it into the wide Gaussian cloud spreads that crowd out a lot: large expansion, high log-likelihood. A rare window has little crowd to spread: lower value.

S_3, idea 4: Hutchinson's trick

Problem: the divergence needs the 768 diagonal entries of a 768×768 Jacobian. One backward pass gives one row, so the exact trace costs 768 passes *every time* the solver evaluates the velocity: far too slow.

Trick: for any square A and a random $\boldsymbol{v}$ with independent entries ± 1 (prob. $\frac{1}{2}$ each, a *Rademacher* vector),

$$\boxed{\mathbb{E}\big[\boldsymbol{v}^\top A\,\boldsymbol{v}\big] = \operatorname{tr}(A)}$$

Why: $\boldsymbol{v}^\top A\boldsymbol{v} = \sum_{j,k} A_{jk} v_j v_k$. If $j = k$: $v_j^2 = 1$ always. If $j \neq k$: $v_j v_k = \pm 1$ with equal probability, average 0. Only the diagonal survives.

Noise: one draw is unbiased but noisy, with variance

$$\operatorname{Var}\big(\boldsymbol{v}^\top A\boldsymbol{v}\big) = \sum_{j<k} \big(A_{jk} + A_{kj}\big)^2,$$

which grows with the off-diagonal entries.

S_3: Hutchinson on a 2×2 example, and why it is cheap

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathrm{tr}(A) = a + d.$$

$\boldsymbol{v}$	$\boldsymbol{v}^\top A \boldsymbol{v}$
$(+1, +1)$	$a + d + (b + c)$
$(-1, -1)$	$a + d + (b + c)$
$(+1, -1)$	$a + d - (b + c)$
$(-1, +1)$	$a + d - (b + c)$

Each value is off by $\pm(b + c)$; the average is exactly $a + d$. Variance $= (b + c)^2$.

Cheap: $\boldsymbol{v}^\top \frac{\partial f}{\partial \boldsymbol{x}} \boldsymbol{v}$ needs **one** backward pass: differentiate the single number $f(\boldsymbol{x}, \sigma) \cdot \boldsymbol{v}$ w.r.t. $\boldsymbol{x}$ to get $\boldsymbol{v}^\top \frac{\partial f}{\partial \boldsymbol{x}}$, then dot with $\boldsymbol{v}$.

As implemented (`ElucidatedDiffusion.log_likelihood`):

- one Rademacher $\boldsymbol{v}$ per window, fixed along the whole path;
- the solver integrates the position $\boldsymbol{x}(\sigma)$ and the running total $\int \boldsymbol{v}^\top \frac{\partial f}{\partial \boldsymbol{x}} \boldsymbol{v} \, d\sigma$ together;
- unconditional, batches of 25 windows, 100 windows in total.

S_3: numbers, strengths and weaknesses

Values are log-densities in the normalised 768-dimensional space: 10th / 50th / 90th percentile 1787 / 2001 / 2114 (range 605–2238). Large and positive because a *density* (not a probability) can exceed 1 when data are concentrated, and 768 such factors add up in the log. Only the order is used.

Strengths

+ the model's *actual* log-density of the whole window (up to solver error and Hutchinson noise)

+ exactly what S_2 approximates: the natural yardstick for S_2

+ uses step order, actions and rewards

Weaknesses

– density of the **model**, not the true data: a reference for S_2, **not a gold standard for support**

– one Hutchinson vector per window: noisy per window

– slow (adaptive ODE solve, backward pass per step): only 100 windows, so correlations with it have wide CIs

– hardest of the three to explain

Do the metrics measure what they should? How we check

No ground-truth "support" exists, so three standard checks:

1. **Convergent validity:** the three metrics agree with each other.
2. **Discriminant validity:** they do not simply repeat quality.
3. **Predictive validity:** they predict what H2a is about (physical validity of edits).

Spearman rank correlation: correlation of the ranks; without ties

$$\rho = 1 - \frac{6\sum_{i=1}^{n} g_i^2}{n(n^2-1)}, \qquad g_i = \mathrm{rank}(u_i) - \mathrm{rank}(w_i).$$

Ranks are right here: metrics are used only as percentiles and are linked by monotone transforms (distance, log-error, log-density).

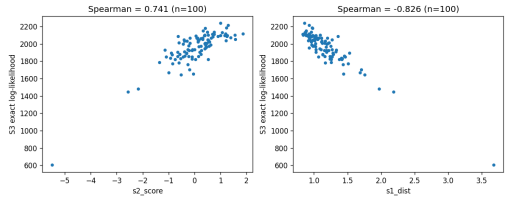
Bootstrap 95% CI: resample the windows with replacement 5,000 times, recompute ρ, keep the middle 95%.

Not the regression coefficients: β_S, β_Q (best μ regressed on S and Q) test the *hypothesis*; they play no role in validating the metrics.

Convergent validity: the three metrics agree

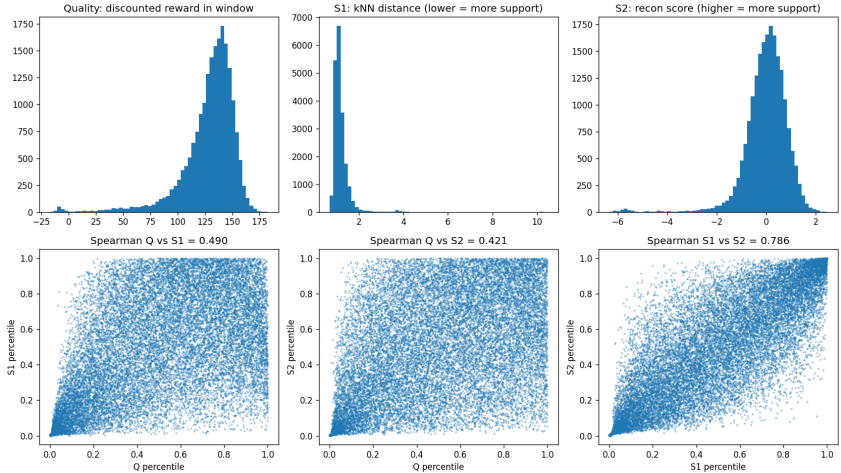
pair	n	ρ	95% CI
S_1 vs S_2	20,000	0.79	narrow
S_2 vs S_3	100	0.74	[0.63, 0.82]
S_1 vs S_3	100	0.83	[0.74, 0.89]
$S_2(\sigma{=}0.16)$ vs S_3	100	0.61	[0.46, 0.74]
$S_2(\sigma{=}0.5)$ vs S_3	100	0.71	[0.59, 0.79]
$S_2(\sigma{=}2.4)$ vs S_3	100	0.57	[0.42, 0.69]

- Three metrics built on different assumptions order windows similarly.
- Combining the three σ (0.74) tracks S_3 better than any single σ.



The 100 windows with S_3. Left: s2_score vs log-likelihood. Right: s1_dist vs log-likelihood (lower distance = more support, so the relation is negative).

Discriminant validity: support is not quality



20,000 windows. Spearman with quality: 0.49 (S_1), 0.42 (S_2). Related (unusual windows earn less) but far from identical, so they can be studied as two factors.

Predictive validity: support predicts which edits break physics

400 windows (5×5 grid of quality and support) edited at 7 strengths and replayed in MuJoCo. **Physics error** = distance between the generated next state and the simulator's next state, in dataset-std units.

μ	0.1	0.2	0.3	0.35	0.4	0.45	0.5
S_2 vs physics error	-0.29	-0.36	-0.34	-0.28	-0.24	-0.18	-0.13
S_1 vs physics error	-0.25	-0.34	-0.32	-0.26	-0.28	-0.25	-0.15
Q vs physics error	$+0.07$	$+0.01$	$+0.01$	$+0.09$	$+0.04$	-0.14	-0.07

Both support metrics predict which edits break physics, at every editing strength. Quality does not.

Three example windows

	ep.	start	q	s1_dist	$e(0.16)$	$e(0.5)$	$e(2.4)$	S_1	S_2
lowest S_2	593	769	-8.2	3.82	0.0107	0.0716	0.518	0.004	0.000
median S_2	738	546	138.9	0.97	0.0049	0.0213	0.135	0.80	0.50
highest S_2	354	552	138.3	0.77	0.0032	0.0145	0.100	0.999	1.00

- The least supported window earns almost no reward: an unusual moment in which the cheetah barely moves forward.
- Its nearest other-run neighbours are $\approx 4\times$ farther away, and the model's reconstruction errors are 2–5$\times$ larger.
- S_1 and S_2 agree on all three windows.

Summary of the three support metrics

	S_1 nearest-neighbour	S_2 reconstruction	S_3 exact log-likelihood
measures	density of the **data** around each step	how well the **model** undoes noise	the **model's** density of the window
looks at	single transitions	whole window	whole window
needs the model	no	yes	yes
cost (20k windows)	seconds (GPU)	≈ 3 s (GPU)	100 windows only
strength	direct, model-free	matches what editing uses	exact for the model
weakness	ignores step order; arbitrary k	model-dependent; heuristic weights	slow, noisy, model-dependent
role	robustness check	**primary metric**	validation of S_2

Conclusions

- **Quality** Q is the discounted reward inside the window: exact.
- **Local support** has no ground truth. Three metrics on different assumptions
  - rank windows similarly ($\rho = 0.74$–0.83),
  - are distinct from quality ($\rho = 0.42$–0.49),
  - predict the physical validity of edits, while quality does not.
- S_3 is a **reference for S_2, not a gold standard**: exact only for the model, noisy per window, 100 windows.
- Correct claim: "consistent with each other and predictive of edit validity", **not** "validated against ground truth".

How a true ground-truth test could be built (not done)

Control support instead of estimating it: remove most data from one region of state space, retrain, and check that S_1, S_2, S_3 fall in that region and only there.

References

- J. Lee, S. Yun, T. Yun, J. Park. GTA: Generative Trajectory Augmentation with Guidance for Offline Reinforcement Learning. NeurIPS 2024.
- T. Karras, M. Aittala, T. Aila, S. Laine. Elucidating the Design Space of Diffusion-Based Generative Models. NeurIPS 2022.
- Y. Song, J. Sohl-Dickstein, D. Kingma, A. Kumar, S. Ermon, B. Poole. Score-Based Generative Modeling through Stochastic Differential Equations. ICLR 2021.
- Y. Song, C. Durkan, I. Murray, S. Ermon. Maximum Likelihood Training of Score-Based Diffusion Models. NeurIPS 2021.
- D. Kingma, T. Salimans, B. Poole, J. Ho. Variational Diffusion Models. NeurIPS 2021.
- M. Hutchinson. A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines. Comm. Statistics, 1989.
- J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. 2020.

Code: `src/h2/quality.py`, `src/h2/support.py` (`knn_distance`, `recon_error`, `log_likelihood`); run `python scripts/h2_run.py pool`.